Problema do caixeiro viajante e simulated annealing: um breve estudo de distâncias, temperaturas e otimização

Caio M. Carregari[1], Clidenor B. Melo[2]

[1]Universidade do Estado de Santa Catarina (UDESC)

Joinville – SC – Brasil CEP: 89.219.710

{caio.mq.carregari@gmail.com, neto.neto0100@gmail.com}

Abstract. *This paper presents the implementation of a Simulated Annealing (SA) algorithm for solving the Traveling Salesman Problem (TSP). Inspired by the physical process of metal cooling, the algorithm uses a temperature parameter that controls the acceptance of worse solutions throughout the process, allowing it to escape local minima and improve the global search. Thus, in this paper, we will present the results of three cooling schedules in instances of 51 and 100 cities, discussing their results in a comparative statistical analysis and, finally, a discussion of possible future work.*

Resumo. *Este trabalho apresenta a implementação de um algoritmo de Simulated Annealing (SA) para a resolução do Problema do Caixeiro Viajante (TSP). Inspirado no processo físico de resfriamento de metais, o algoritmo utiliza um parâmetro de temperatura que controla a aceitação de soluções piores ao longo do processo, permitindo escapar de mínimos locais e melhorar a busca global. Assim, neste trabalho, apresentaremos o resultado de três cooling schedules em instâncias de 51 e 100 cidades, discutindo seus resultados em uma análise estatística comparativa e, por fim, uma discussão de possíveis trabalhos futuros.*

1. Introdução

O Problema do Caixeiro Viajante (*Travelling Salesman Problem* – TSP) foi estudado por volta de 1930 pelo matemático austríaco Karl Menger. O problema consiste em encontrar o caminho mais curto que visite cada cidade de uma lista exatamente uma vez e retorne à cidade de origem. Menger observou que o problema poderia ser resolvido por força bruta, mas mesmo aplicando heurísticas simples, como a do "vizinho mais próximo", a solução obtida não era ótima. Desde então, diversas heurísticas foram propostas com o objetivo de aproximar o resultado para o ótimo e, em 1983, Kirkpatrick et al. apresentaram a heurística denominada *Simulated Annealing* (SA).

O trabalho *Optimization by Simulated Annealing*, de Kirkpatrick et al. (1983), discute o processo de dilatação e resfriamento de metais e como esse comportamento físico pode ser aplicado a problemas de otimização. Segundo os autores, "essa conexão com a mecânica estatística revela novas informações e oferece uma perspectiva diferente sobre os problemas e métodos tradicionais de otimização". Assim, o algoritmo é fundamentado no conceito de temperatura, que controla o grau de exploração do espaço de busca.

Em temperaturas elevadas – quando o metal apresenta maior dilatação – o algoritmo tende a aceitar soluções piores com maior frequência, favorecendo uma exploração mais ampla e evitando mínimos locais. Dessa forma, à medida que a temperatura diminui – simulando o resfriamento e a contração do metal – a probabilidade de aceitar soluções piores é reduzida, levando o sistema a uma convergência gradual em direção a uma solução próxima do ótimo global.

Nesse contexto, esse trabalho aplica o conceito de *Simulated Annealing* (SA) para resolver o problema do caixeiro viajante (TSP), discutindo três fórmulas de resfriamento de temperatura, e realizando uma análise estatística sobre os resultados obtidos.

2. Metodologia e desenvolvimento

O desenvolvimento desse trabalho foi feito utilizando a linguagem de programação *Python,* utilizando o VSCode como ambiente de programação. A premissa consiste em carregar duas instâncias de cidades, uma de 51 cidades (eil51) e outra de 100 cidades (kroA100). Cada instância representa um conjunto de coordenadas cartesianas (x, y) correspondentes à localização das cidades, sendo utilizada para calcular a matriz de distâncias euclidianas entre os pontos.

Com a matriz de distâncias previamente calculada, inicia-se a resolução do problema do caixeiro viajante. Primeiramente, uma cidade inicial é selecionada aleatoriamente e, por meio de uma heurística gulosa, o algoritmo percorre o caminho escolhendo sempre o vizinho mais próximo até que todas as cidades tenham sido visitadas exatamente uma vez. Os índices das cidades visitadas são então armazenados em uma lista, que serve como solução inicial para o *Simulated Annealing*. A partir desse ponto, o algoritmo introduz perturbações controladas nessa solução, essenciais para escapar de mínimos locais e permitir uma exploração efetiva do espaço de busca. Entre as perturbações utilizadas, destacam-se:

- Troca simples (swap): duas cidades têm suas posições invertidas, gerando pequenas alterações estruturais no percurso (o número de pares trocados por RUN varia de 1 a 5);
- 2-Opt: proposta por Croes (1958), é um algoritmo de otimização que inverte um segmento de um caminho. *Exemplo:* dada a rota inicial 1 → 2 → 4 → 3, a aplicação do 2-Opt produz 1 → 2 → 3 → 4.

Como foi mencionado anteriormente, para realizar o resfriamento do *Simulated Annealing,* foram escolhidas três fórmulas, sendo elas:

- *Cooling Schedule* 0 (Linear): $Ti = T0 - i \cdot \frac{T0 - Tn}{N}$
- *Cooling Schedule* 2: $|Ti = \frac{A}{i+1} + B|\ |A = \frac{(T0-Tn)\cdot(N+1)}{N}|\ |B = T0 - A|$
- *Cooling Schedule 4* (Sigmoid): $Ti = \frac{T0 - Tn}{1 + e^{0.00003\cdot(i - N/2)}} + Tn$

Observação: Conforme discutido com o professor em sala, a constante da fórmula de *Cooling Schedule* 4 (0.3) foi ajustada para 0.00003, a fim de evitar estouros numéricos (overflow) durante as iterações do código.

Ainda, a seguir, segue a lista sobre o que cada operando significa:

- Ti: Representa a temperatura na iteração atual;
- i: Representa o número da iteração;
- $T0$: Representa a temperatura inicial (detalhada na Seção 3);
- Tn: Representa a temperatura final (detalhada na Seção 3);
- N: Representa o número máximo de iterações (detalhada na Seção 3).

Por fim, para cada instância foram realizadas 10 execuções independentes (RUNs), de modo a permitir uma análise estatística sobre os resultados. Assim, os dados de temperatura, distância atual e melhor distância em cada iteração foram registrados e, posteriormente, utilizados para a geração de gráficos comparativos e interpretação dos resultados obtidos.

3. Experimentos e resultados obtidos

Para visualizar nossos resultados – o código foi compilado no terminal do Linux (Ubuntu) – utilizamos a biblioteca *Matplotlib,* tanto para visualizar o gráfico da melhor RUN e sua temperatura – com o progresso da distância e da temperatura ao longo das iterações – quanto para o gráfico gerado da análise estatística no modelo boxplot.

Dessa forma, para a instância de 51 cidades, de forma empírica, definimos os seguintes parâmetros para realizar nossos testes:

- $T0$ = 50
- $ITMAX$ = 500000
- TN = 0.001
- $RUNS = 10$

Ainda é importante destacar que cada execução (RUN) realiza o número total de iterações definido por ITMAX, ou seja, cada uma das 10 RUNS percorreu 500.000 iterações. Assim, o processo completo correspondeu a 5 milhões de iterações, garantindo uma amostragem ampla para a análise estatística dos resultados.

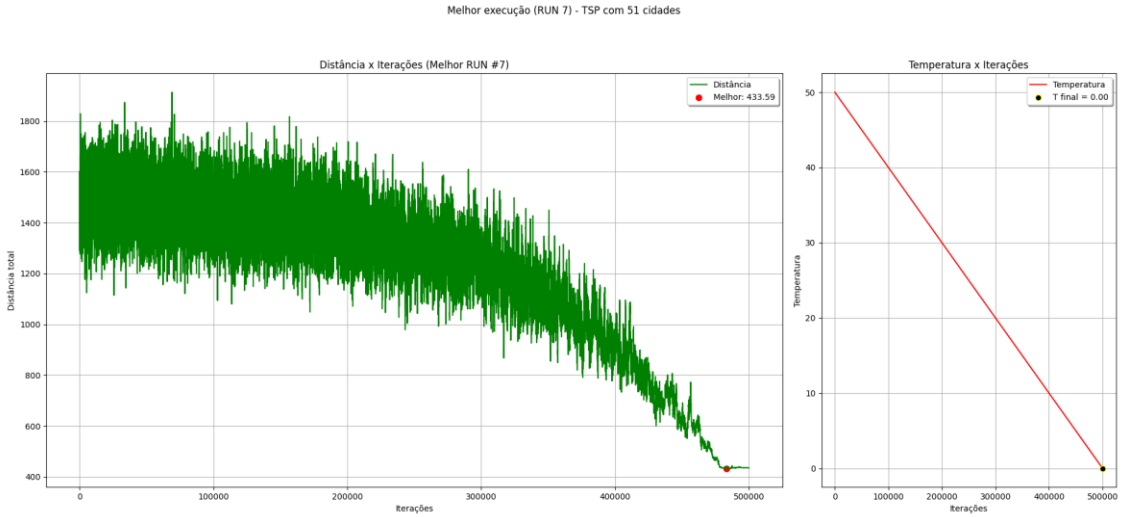

Figura 1. Gráfico da distância e temperatura da fórmula de resfriamento 0 (Linear) – 51 cidades

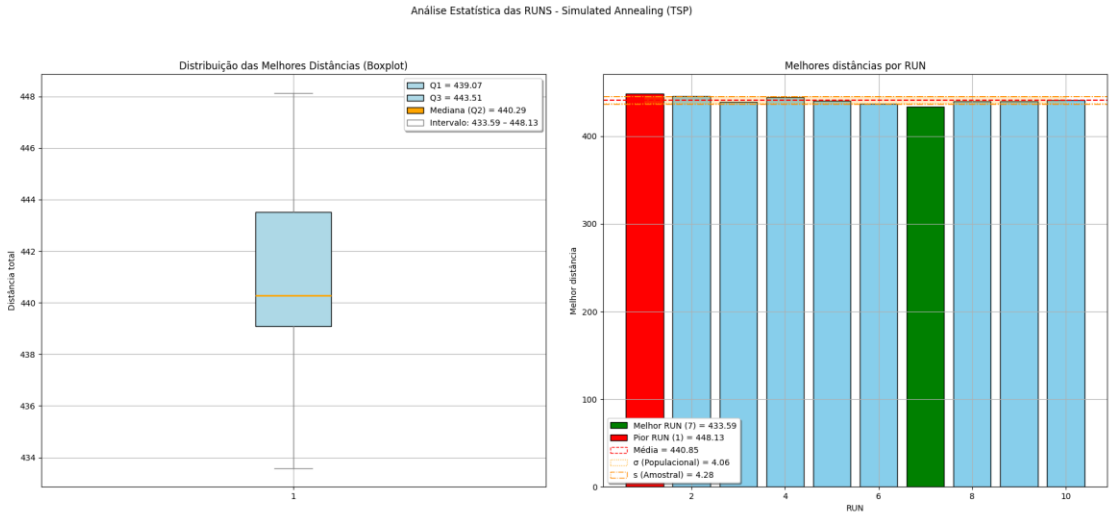

Figura 2. Análise estatística da fórmula de resfriamento 0 (Linear) – 51 cidades

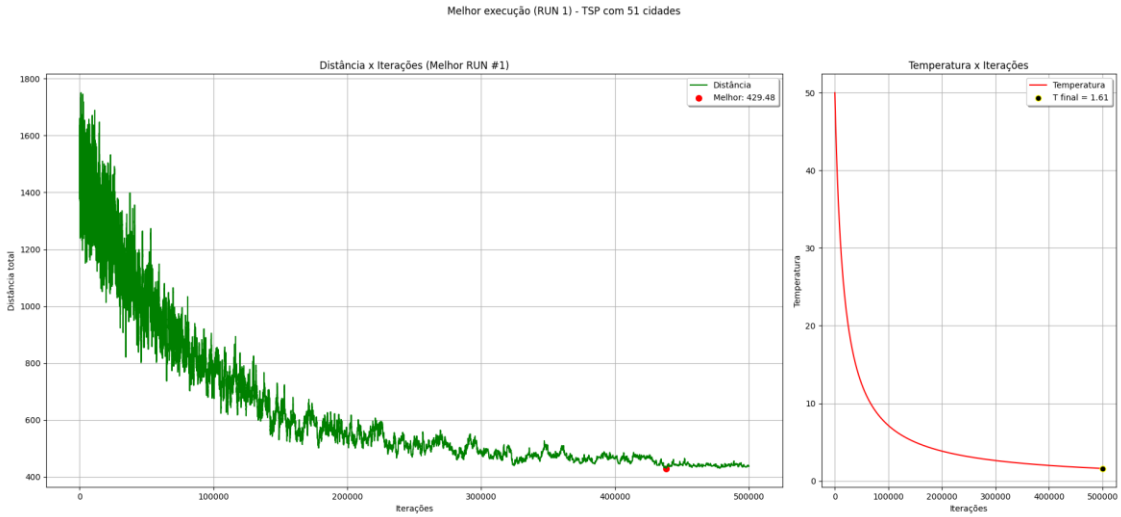

Figura 3. Gráfico da distância e temperatura da fórmula de resfriamento 2 – 51 cidades

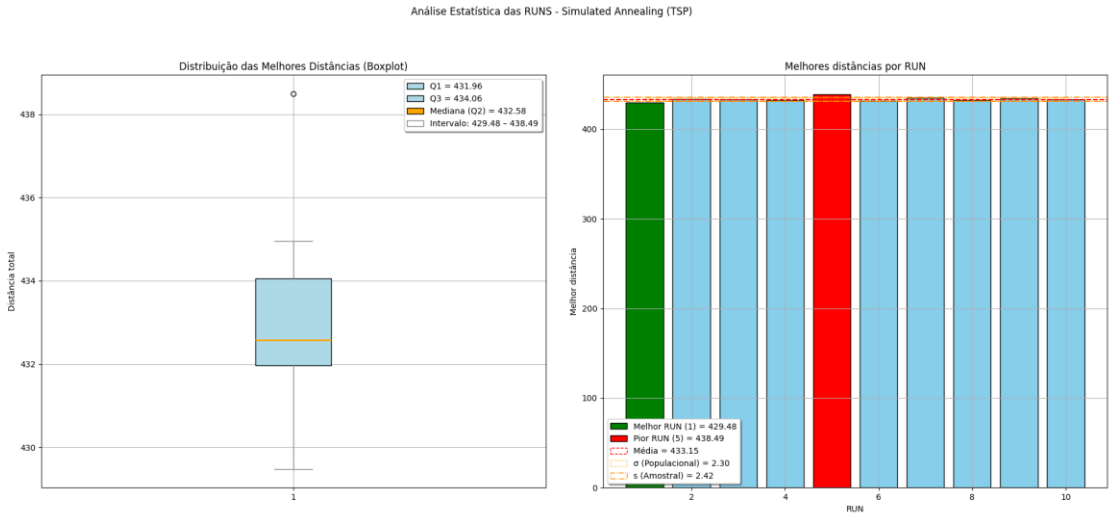

Figura 4. Análise estatística da fórmula de resfriamento 2 – 51 cidades

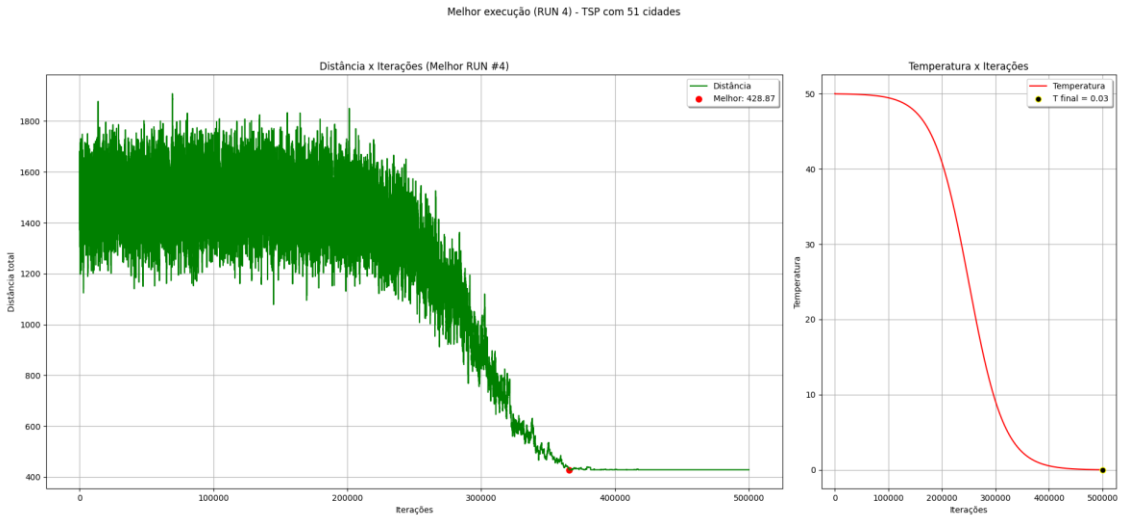

Figura 5. Gráfico da distância e temperatura da fórmula de resfriamento 4 (Sigmoid) – 51 cidades

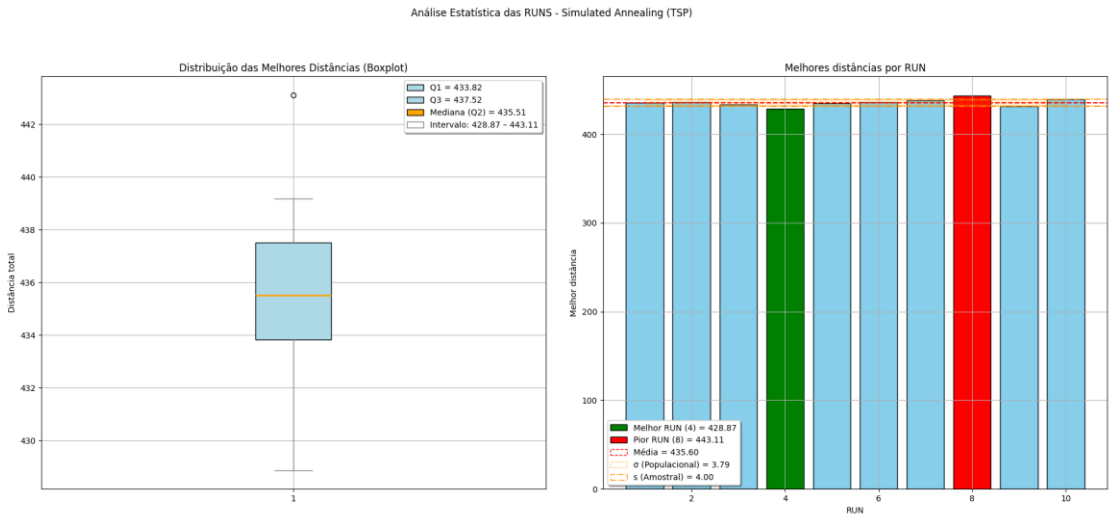

Figura 6. Análise estatística da fórmula de resfriamento 4 (Sigmoid) – 51 cidades

Com os gráficos feitos, podemos observar que as três fórmulas garantiram um resultado muito próximo do resultado ótimo (426). Uma análise mais profunda dos resultados será feita na Seção 4 desse trabalho.

Dessa forma, seguiremos o mesmo padrão para a instância de 100 cidades, onde utilizamos os seguintes parâmetros, que também foram definidos de forma empírica:

- $T0 = 2000$
- $ITMAX = 500000$
- $TN = 0.001$
- $RUNS = 10$

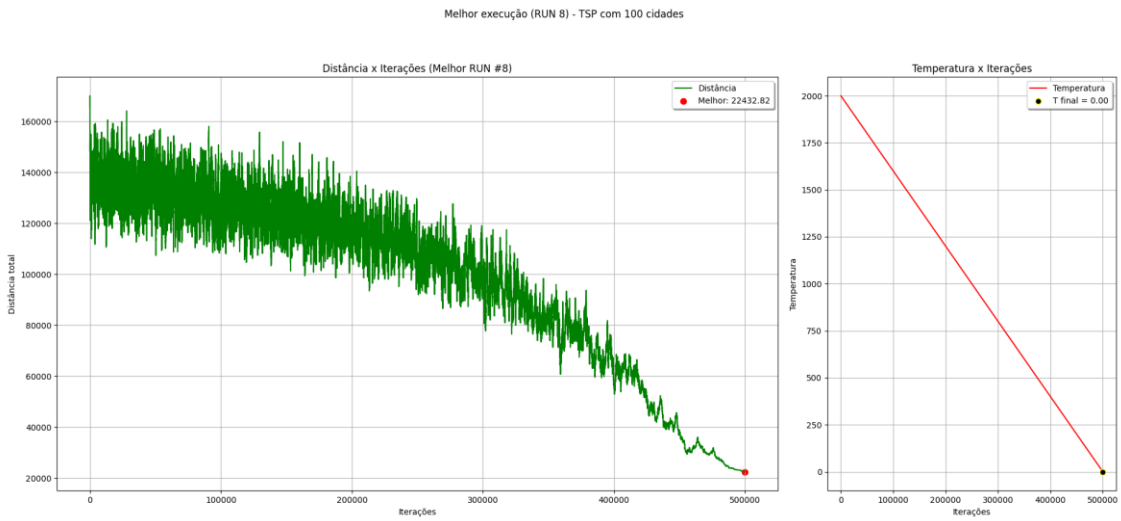

Figura 7. Gráfico da distância e temperatura da fórmula de resfriamento 0 (Linear) – 100 cidades

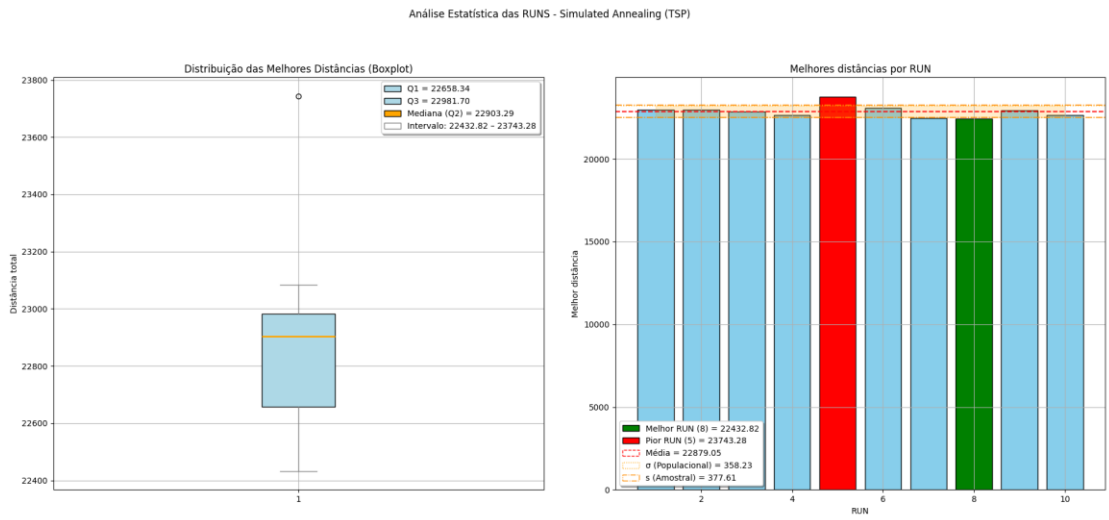

Figura 8. Análise estatística da fórmula de resfriamento 0 (Linear) – 100 cidades

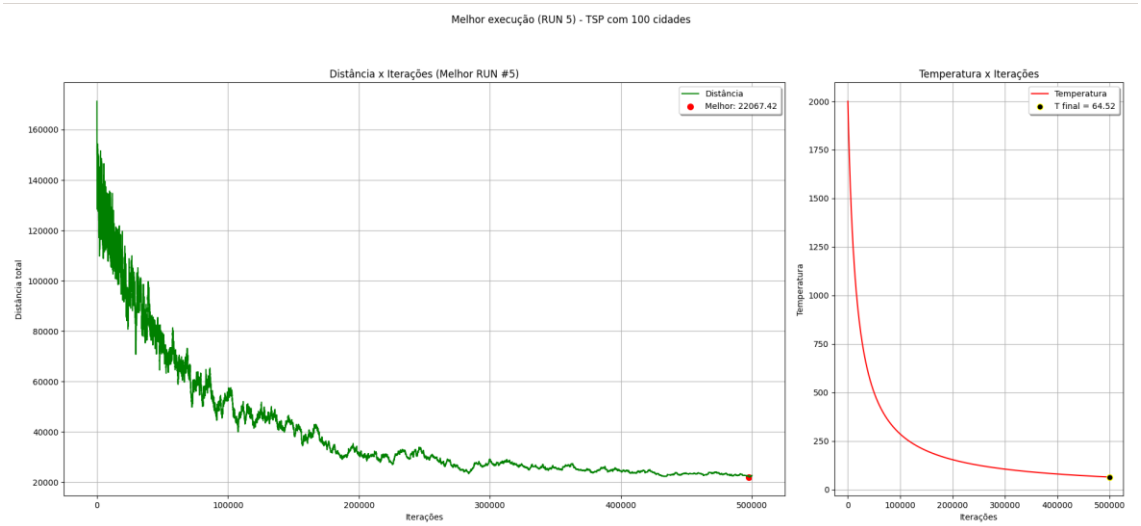

Figura 9. Gráfico da distância e temperatura da fórmula de resfriamento 2 – 100 cidades

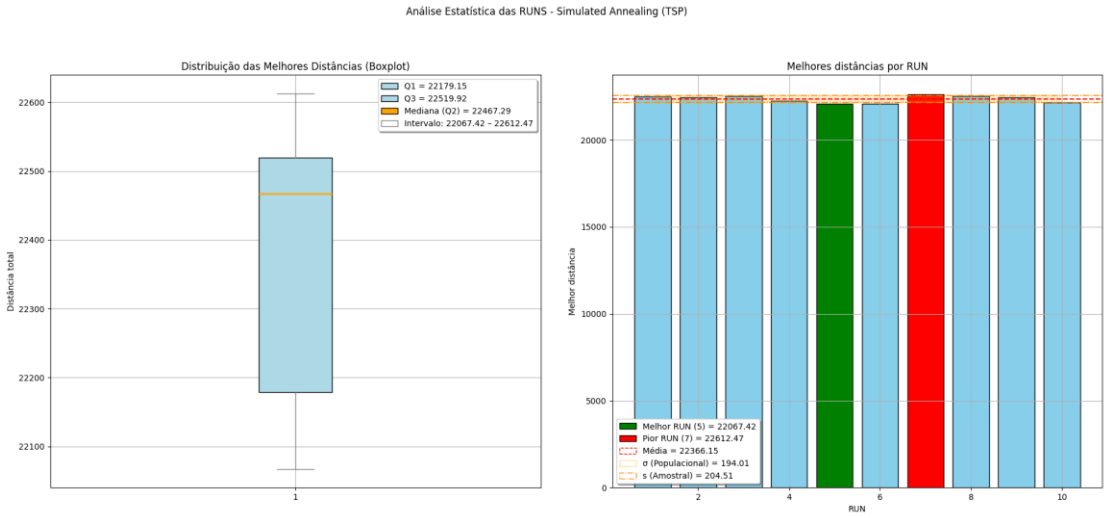

Figura 10. Análise estatística da fórmula de resfriamento 2 – 100 cidades

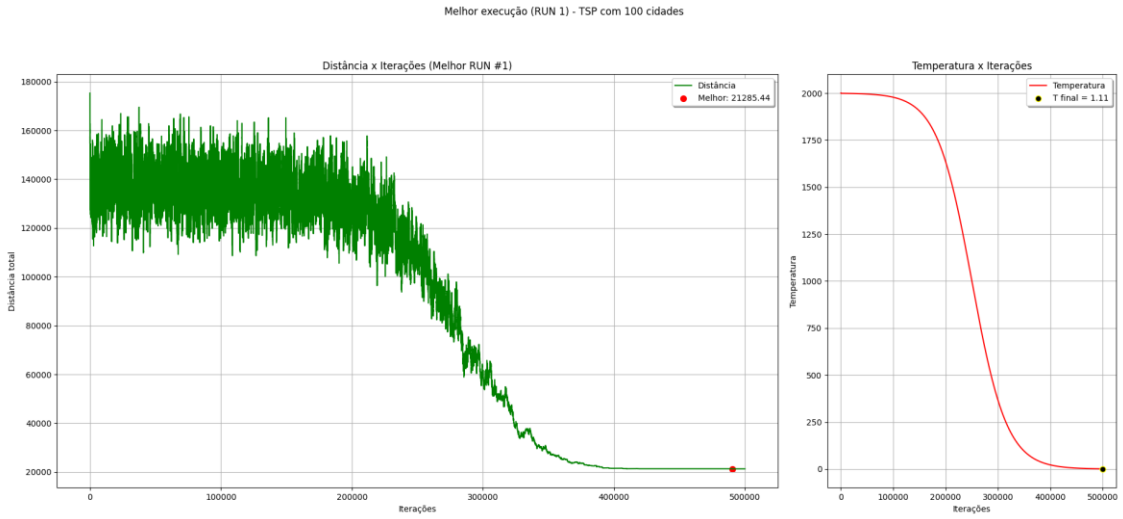

Figura 11. Gráfico da distância e temperatura da fórmula de resfriamento 4 (Sigmoid) – 100 cidades

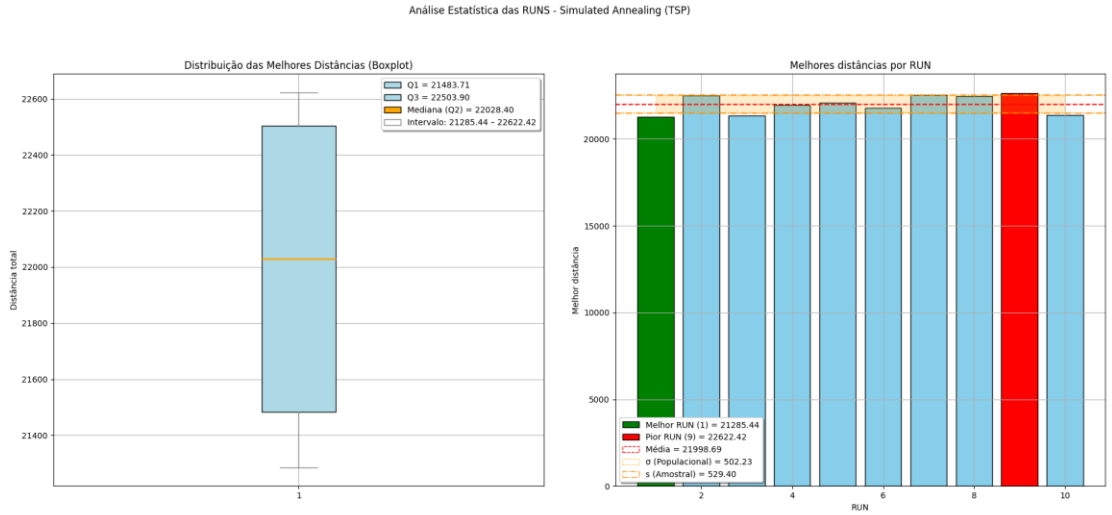

Figura 12. Análise estatística da fórmula de resfriamento 4 (Sigmoid) – 100 cidades

Assim, com o ajuste de parâmetros para a instância de 100 cidades, também notamos que todas as três fórmulas estão bem próximas do ótimo (21.282).

4. Análise dos resultados

Em uma análise geral, os resultados obtidos em ambas as instâncias, se demonstraram extremamente satisfatórios. Dessa forma, podemos notar no gráfico estatístico uma baixa variabilidade entre os resultados, com um desvio padrão reduzido, indicando que o algoritmo em si manteve um comportamento estável e previsível. Na primeira instância, 51 cidades, a fórmula que mais chegou perto do ótimo foi a *Cooling Schedule* 4 (Sigmoid) – com diferença de 2,87 em relação ao valor ótimo – que se destacou por apresentar um processo de resfriamento gradual e equilibrado, permitindo uma boa exploração do espaço de soluções nas iterações iniciais e uma convergência mais controlada nas etapas finais. Por outro lado, ao compararmos com a *Cooling Schedule* 0 que, por ser uma fórmula de resfriamento linear, apresentou o pior resultado, tendo uma diferença de 7,69 comparado ao ótimo.

Na segunda instância, com 100 cidades, a fórmula que novamente apresentou o melhor desempenho foi a *Cooling Schedule* 4 (Sigmoid), com uma diferença de 3,44 em relação ao valor ótimo. Em contrapartida, a *Cooling Schedule* 0 manteve o pior resultado, apresentando uma diferença de 1.150,82. Mesmo diante de um aumento significativo no número de cidades, o algoritmo demonstrou ser robusto e consistente, mantendo a qualidade das soluções obtidas.

4.1. Tabela comparativa

Tabela 1. Comparação dos resultados da melhor distância, com o resultado ótimo esperado

	51 Cidades	100 Cidades	Distância do ótimo	
			51 Cidades	100 Cidades
Cooling Schedule 0	433,59	22.432,82	7,69	1.150,82
Cooling Schedule 2	429,48	22.067,42	3,48	785,42
Cooling Schedule 4	428,87	21.285,44	2,87	3,44

5. Conclusão e futuras pesquisas

Neste trabalho, foi realizada a implementação de um algoritmo para resolver o problema do caixeiro viajante (TSP), com a otimização de uma técnica denominada *Simulated Annealing* que, por meio de temperaturas, melhora a capacidade de busca do algoritmo.

A priori, a primeira fase do projeto conta com a instância eli51, composta por 51 cidades com suas respectivas coordenadas em um plano cartesiano, servindo como base para os parâmetros de temperatura inicial, temperatura final e número máximo de iterações. Em seguida, após ajustes nos parâmetros, foi analisada a instância com 100 cidades (kroA100), permitindo verificar o comportamento do algoritmo em um cenário de maior complexidade.

Durante os experimentos, escolhemos três fórmulas principais para reger o comportamento da temperatura em nosso algoritmo. Assim, ao final da análise, percebemos que a temperatura com maior êxito foi a *Cooling Schedule* 4, Sigmoid, apresentando boa exploração e convergência.

De modo geral, o *Simulated Annealing* se mostrou eficiente para o TSP, fornecendo soluções próximas do ótimo mesmo em instâncias maiores. Além disso, observou-se que a escolha adequada da função de resfriamento tem impacto direto na qualidade e estabilidade dos resultados. Como trabalho futuro, seria interessante novos meios de perturbação, além da troca de pares e 2-Opt, e ainda testar novas fórmulas de resfriamento para obter resultados ainda melhores e, talvez, superar o ótimo. Em última análise, este estudo mostra que até mesmo um fenômeno físico – a dilatação e o resfriamento de um metal – pode inspirar a lógica de um algoritmo, unindo o comportamento natural ao pensamento computacional.

Referências

Bertsimas, D. and Tsitsiklis, J. *Simulated Annealing*. *Statistical Science*, v. 8, n. 1, p. 10–15, 1993. DOI: 10.1214/ss/1177011077.

Kirkpatrick, S., Gelatt JR, C. D. and Vecchi, M. P. *Optimization by Simulated Annealing*. *Science*, v. 220, n. 4598, p. 671–680, 1983.

Luke, B. T. *Simulated Annealing Cooling Schedules*. Disponível em: <http://www.btluke.com/simanf1.html>. Acesso em: 8 nov. 2025.

Croes, G. A. *A Method for Solving Traveling-Salesman Problems*. Operations Research, v. 6, n. 6, p. 791–812, 1958.